Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

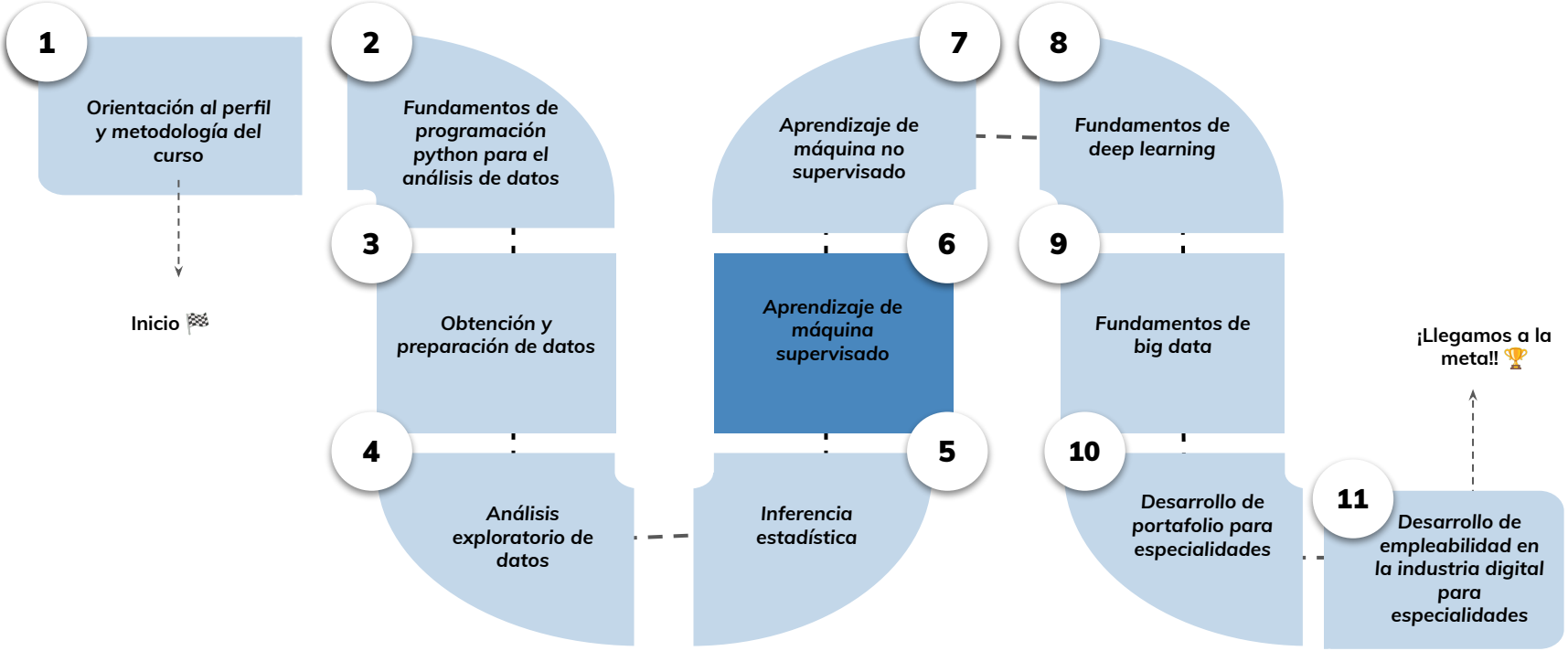
Clase grabada

Optimización de un modelo- Parte I

Aprendizaje Esperado 7: Aplicar técnicas de optimización de un modelo de aprendizaje para la mejora de su desempeño acorde a las buenas prácticas de la industria.

Hoja de ruta

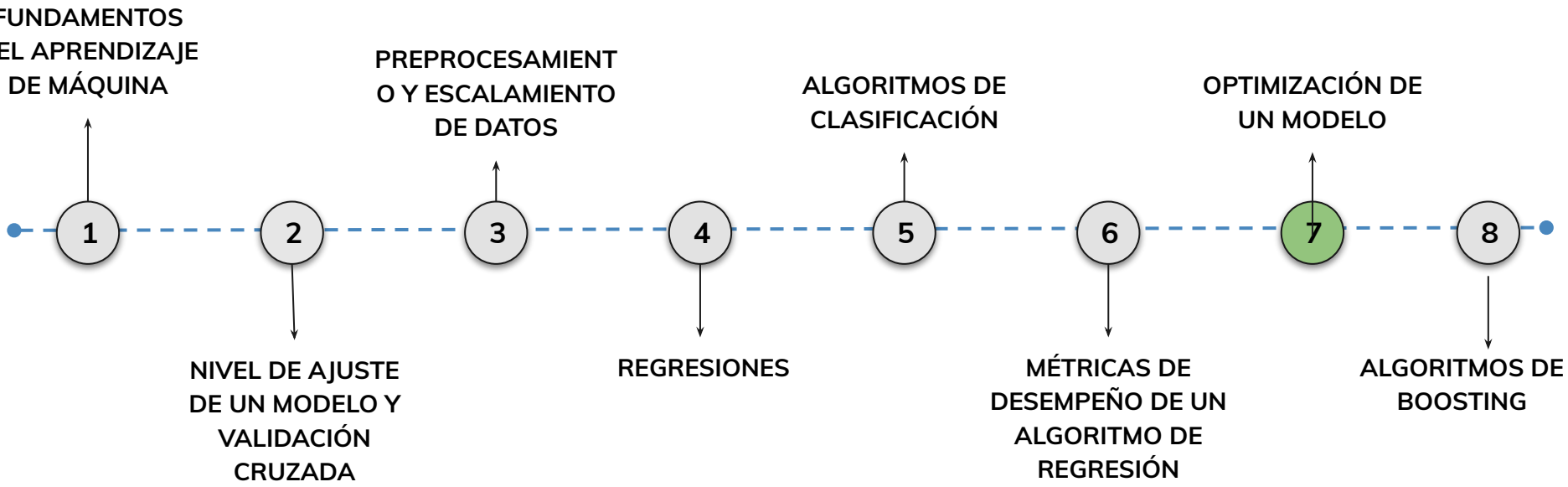
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

7.

¿Qué significa optimizar un modelo?

Comprenderemos cómo mejorar el rendimiento de un modelo ajustando su configuración y seleccionando variables adecuadas.

- **Evaluación del rendimiento inicial**
  - **Establecer una línea base**
  - **Métricas de comparación**
- **Feature Engineering**
  - **Selección de variables más relevantes**
  - **Creación y transformación de atributos**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender la importancia de la optimización de modelos en el flujo de trabajo de machine learning.
- Aplicar técnicas de selección y transformación de variables (Feature Engineering).
- Implementar ajustes de hiperparámetros usando Grid Search y Random Search.
- Usar métodos de regularización (Lasso, Ridge, ElasticNet) para prevenir el sobreajuste.
- Emplear técnicas de balanceo de clases y su impacto en modelos predictivos.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Las principales métricas para evaluar algoritmos de regresión: MAE, MSE, RMSE y R^2.*

→ *Cómo interpretar esas métricas en diferentes contextos.*

→ *Validación cruzada con K-Fold para mejorar la robustez de la evaluación.*

→ *Métricas de clasificación complementarias: accuracy, precisión, recall, F1-score, AUC.*

Optimización de un modelo

MACHINE LEARNING
MODEL OPTIMIZATION

Optimización de Modelos de Aprendizaje Automático

En el desarrollo de modelos de aprendizaje automático, obtener buenos resultados no solo depende del algoritmo elegido, sino también de los mecanismos utilizados para optimizar su desempeño. Estos mecanismos incluyen desde la ingeniería de características hasta el ajuste fino de hiper parámetros, todos aplicados bajo buenas prácticas propias de la industria. En esta lección abordaremos herramientas clave para potenciar modelos predictivos mediante técnicas consolidadas y modernas.

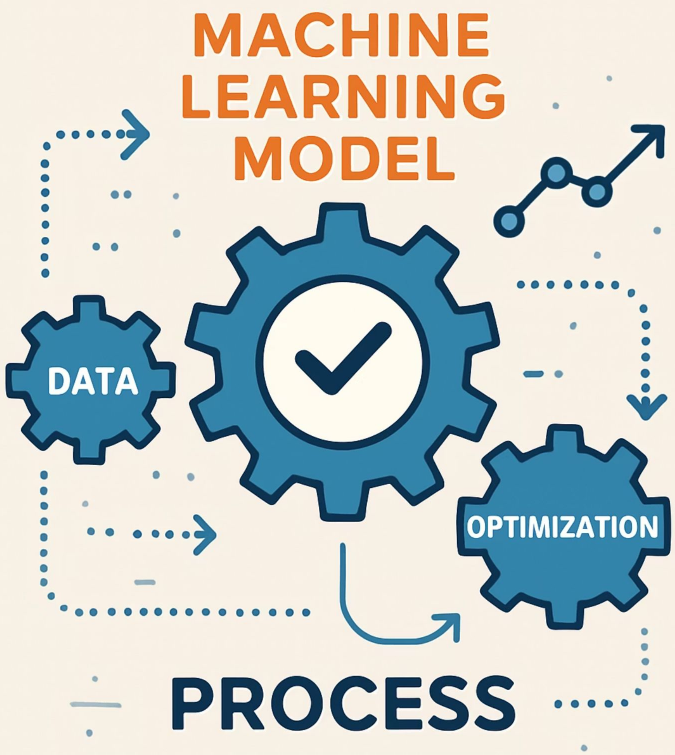
alkemy

Importancia de la Optimización

Comprender estos elementos es fundamental para asegurar que los modelos generalicen adecuadamente sobre datos nuevos. Muchas veces, los errores que se presentan en entornos productivos provienen de un proceso de entrenamiento que no consideró adecuadamente estos mecanismos. Optimizar modelos no es un lujo, sino una necesidad operativa.

OPTIMIZACIÓN DE UN MODELO

La optimización de modelos es un proceso sistemático que busca mejorar el rendimiento predictivo mediante diversas técnicas y ajustes. Este proceso es fundamental para crear modelos que no solo funcionen bien con datos de entrenamiento, sino que también generalicen correctamente a datos nuevos.



alkemy

Mecanismos para mejorar el desempeño de un algoritmo

Preprocesamiento de datos

Mejorar un modelo predictivo no solo consiste en seleccionar el algoritmo más sofisticado. Las prácticas habituales incluyen el preprocesamiento de datos, el diseño de nuevas características, la selección y ajuste de hiperparámetros, el uso de regularización y la validación cruzada. Todas estas técnicas tienen un impacto directo en la capacidad de generalización del modelo.

Preparación de datos

En muchos casos, una buena preparación de los datos puede tener un efecto más relevante que cambiar de algoritmo. Por ejemplo, tratar valores atípicos o normalizar los datos puede hacer que un modelo lineal tenga un rendimiento similar al de modelos más complejos.

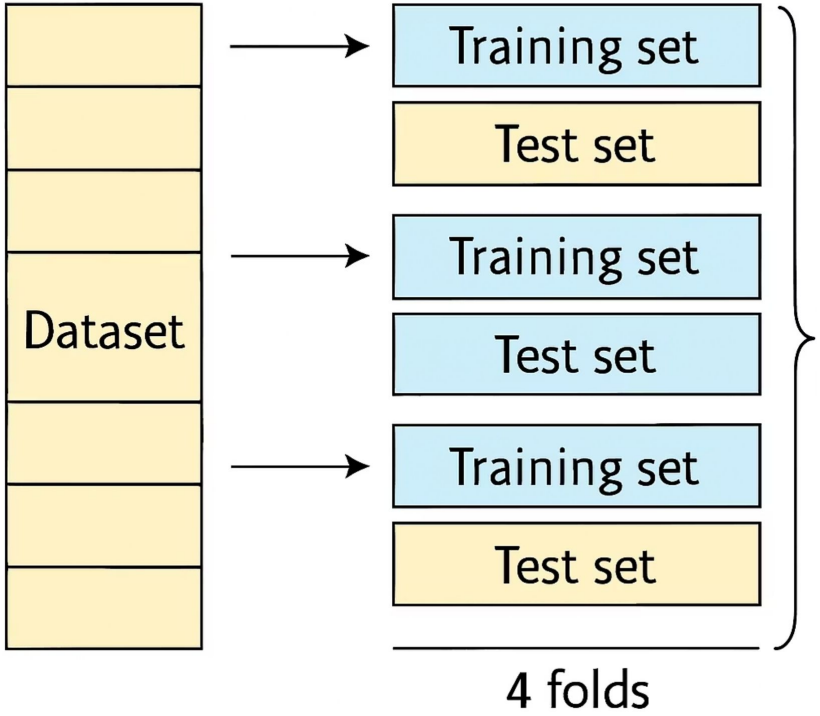
Reducción de dimensionalidad

Otro mecanismo importante es la reducción de dimensionalidad. Técnicas como PCA (Análisis de Componentes Principales) ayudan a eliminar redundancia en los datos, mejorando la eficiencia del modelo y reduciendo el riesgo de sobreajuste.

Métodos de validación y automatización

La selección adecuada del método de validación también mejora el rendimiento. Validación cruzada estratificada o k-fold permiten estimar de forma más robusta el desempeño fuera de la muestra.

Finalmente, es clave automatizar y documentar todos estos procesos mediante pipelines reproducibles. Esto asegura que las mejoras sean sistemáticas y auditables.



¿Qué es Feature Engineering?

El Feature Engineering o ingeniería de características es el proceso de seleccionar, transformar o crear variables predictoras (features) a partir de datos en bruto. Este proceso es esencial para mejorar la capacidad de aprendizaje de los algoritmos y, por ende, el desempeño del modelo final. Un modelo sencillo con buenas variables puede superar con creces a uno complejo entrenado con datos irrelevantes o mal representados.

Componentes del Feature Engineering

Selección de variables relevantes

Uno de los pasos fundamentales en el Feature Engineering es la selección de variables relevantes. No todas las columnas de un conjunto de datos aportan información útil para el modelo; algunas pueden incluso introducir ruido. Seleccionar cuidadosamente las características más representativas ayuda a reducir la complejidad y mejora la interpretabilidad del modelo.

Transformación de variables

Otro componente clave es la transformación de variables. Esto incluye desde operaciones matemáticas simples (como logaritmos o potencias) hasta técnicas más complejas como la codificación de variables categóricas. Por ejemplo, técnicas como One Hot Encoding o Label Encoding permiten convertir datos categóricos en un formato numérico comprensible por los algoritmos.

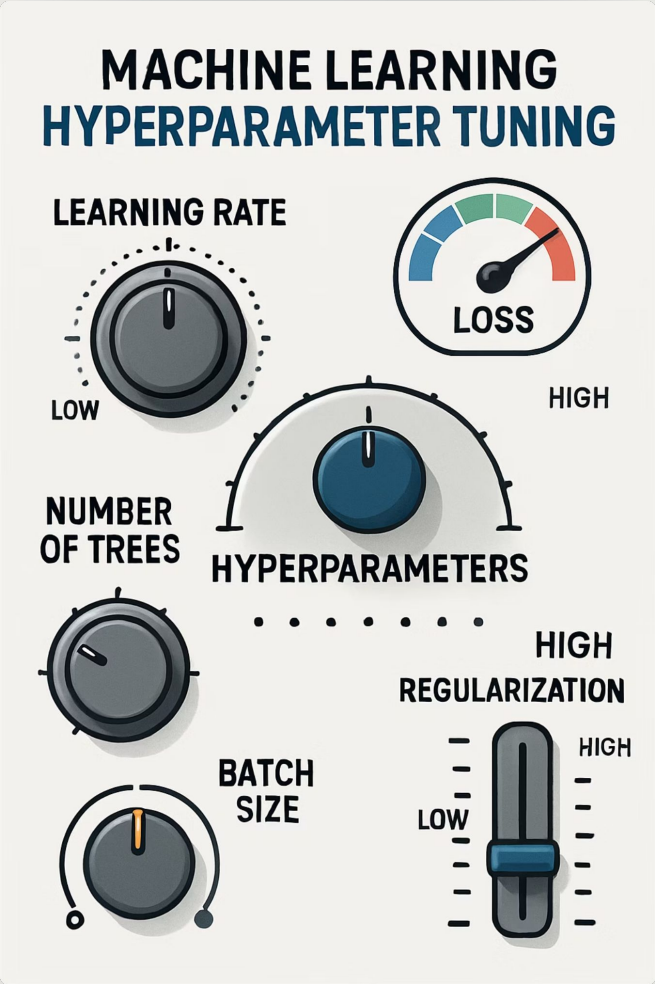
Creación de nuevas variables

También es común la creación de nuevas variables mediante la combinación o interacción de otras. Por ejemplo, en un modelo que analiza préstamos bancarios, una variable como "ingreso por miembro del hogar" (ingreso total dividido entre número de personas) puede ofrecer más valor predictivo que usar esas dos variables por separado.

Escalado y normalización

Finalmente, es importante escalar o normalizar las variables numéricas, especialmente si se usarán algoritmos sensibles a la magnitud de los datos, como los modelos lineales o de distancia (SVM, KNN).

El Feature Engineering requiere creatividad, conocimiento del dominio y prueba continua, y su correcta aplicación marca la diferencia entre un modelo mediocre y uno excelente.



¿Qué es un Hiper Parámetro?

Un **hiper parámetro** es un valor que se configura antes del proceso de entrenamiento de un modelo de aprendizaje automático y que influye en cómo se entrena ese modelo. A diferencia de los **parámetros del modelo**, que se aprenden directamente de los datos durante el entrenamiento (como los coeficientes de una regresión), los hiper parámetros son establecidos manualmente o mediante técnicas de búsqueda automatizada.

alkemy

Ejemplos de hiper parámetros

1

Árboles de decisión

Por ejemplo, si usamos un árbol de decisión, podemos definir hiper parámetros como max_depth (profundidad máxima del árbol), min_samples_split (mínimo número de muestras para dividir un nodo), o criterion (la función para medir la calidad de una división). Estos valores no se ajustan solos; deben probarse distintas combinaciones para encontrar los que ofrecen el mejor rendimiento.

2

Control de ajuste

La selección adecuada de hiper parámetros es crítica para controlar el equilibrio entre **subajuste (underfitting)** y **sobreajuste (overfitting)**. Si un árbol tiene una profundidad muy baja (max_depth=1), es probable que no aprenda correctamente; si es demasiado profundo, podría aprender hasta el ruido del conjunto de entrenamiento, perdiendo capacidad de generalización.

3

Automatización

Para automatizar la selección de hiper parámetros se utilizan técnicas como GridSearchCV o RandomizedSearchCV. Estas herramientas integradas en Scikit-learn permiten evaluar múltiples combinaciones mediante validación cruzada y seleccionar la mejor configuración de forma sistemática.

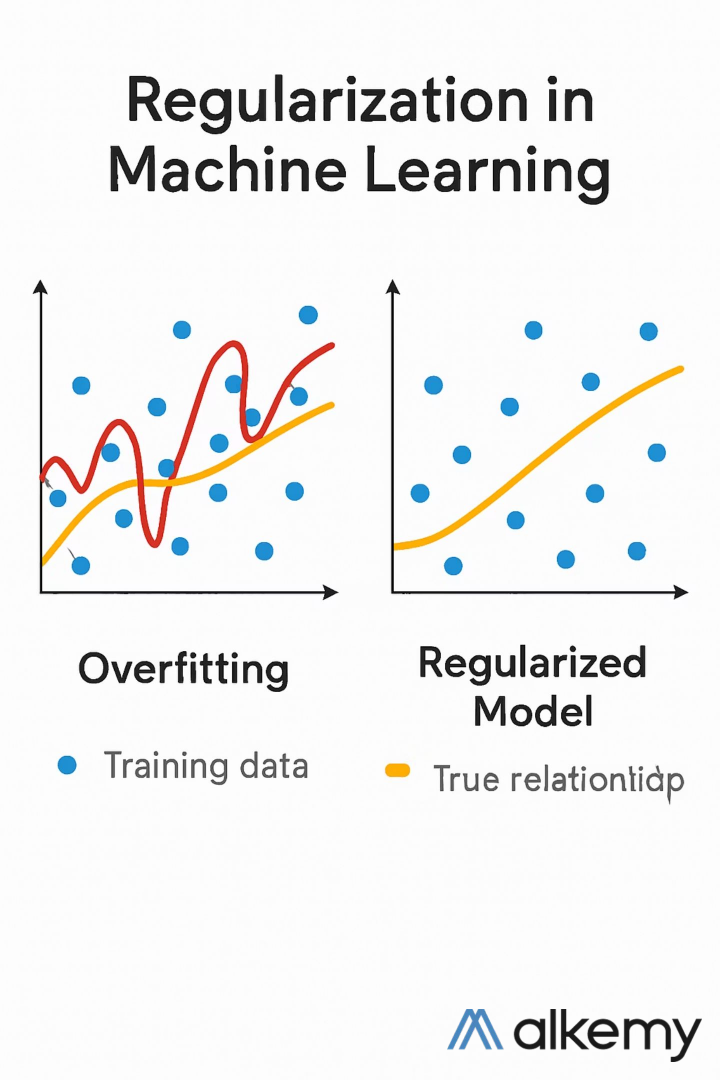
Ejemplo práctico con GridSearchCV

Este ejemplo demuestra cómo una correcta búsqueda de hiperparámetros puede mejorar el rendimiento de un modelo de manera reproducible. Es una práctica estándar en entornos profesionales y clave para la producción de modelos robustos.

```
from sklearn.model_selection import GridSearchCVfrom sklearn.tree import DecisionTreeClassifierfrom
sklearn.datasets import load_irisfrom sklearn.model_selection import train_test_split# Cargar datosX, y =
load_iris(return_X_y=True)X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)# Definir
modelo y grilla de hiperparámetrostree = DecisionTreeClassifier(random_state=42)param_grid = { 'max_depth': [3, 5,
10, None], 'min_samples_split': [2, 5, 10], 'criterion': ['gini', 'entropy']}# Búsqueda en grilla con validación
cruzadagrid_search = GridSearchCV(tree, param_grid, cv=5, scoring='accuracy')grid_search.fit(X_train, y_train)#
Mejores hiperparámetros y rendimientoprint(f"Mejores parámetros: {grid_search.best_params_}")print(f"Mejor
puntuación: {grid_search.best_score_:.4f}")
```

¿Qué es la Regularización?

La **regularización** es una técnica crucial en el aprendizaje automático que se utiliza para evitar el **sobreajuste** (overfitting), una situación en la que el modelo se ajusta demasiado bien a los datos de entrenamiento, pero falla al generalizar a datos nuevos. En términos simples, la regularización penaliza la complejidad del modelo, obligándolo a mantener sus parámetros bajo control.

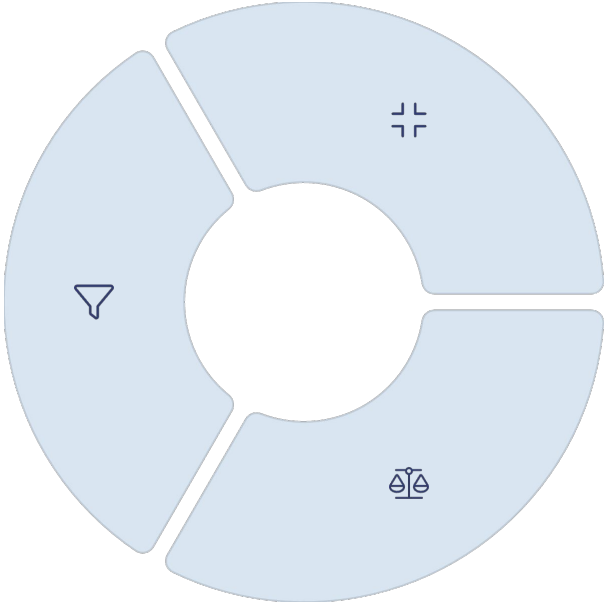


alkemy

Tipos de regularización

L1 (Lasso)

Tiende a reducir a cero los coeficientes menos relevantes, por lo que también actúa como un método de selección de variables.



L2 (Ridge)

Distribuye la penalización entre todos los coeficientes, reduciendo su magnitud pero sin eliminarlos por completo.

Elastic Net

Combinación de L1 y L2, útil cuando se tienen muchas variables correlacionadas y se quiere un balance entre las ventajas de ambas penalizaciones.

alkemy

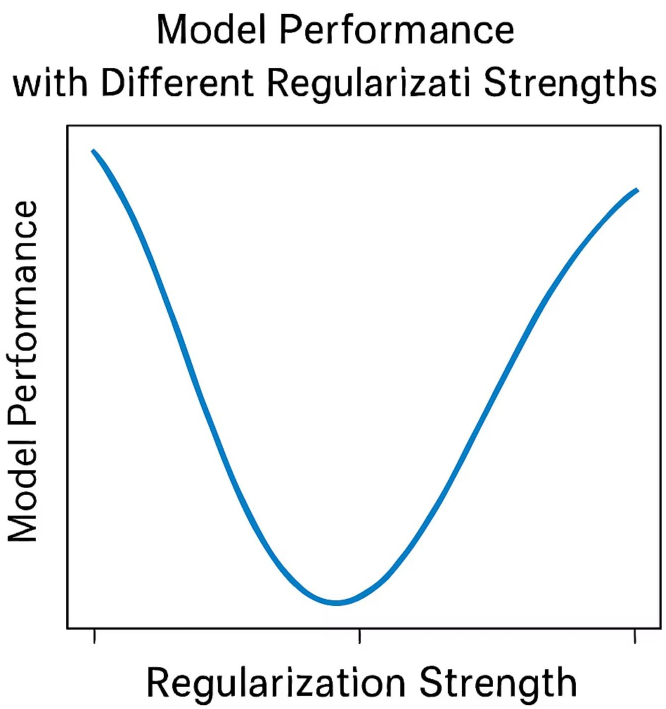
Elección del tipo de regularización

La elección entre L1, L2 o Elastic Net depende del problema. Si se sospecha que solo algunas variables son útiles, L1 es ideal. Si hay muchas variables ligeramente relevantes, L2 puede ser mejor. Elastic Net es útil cuando se tienen muchas variables correlacionadas y se quiere un balance entre las ventajas de ambas penalizaciones.

Implementación de regularización en Scikit-learn

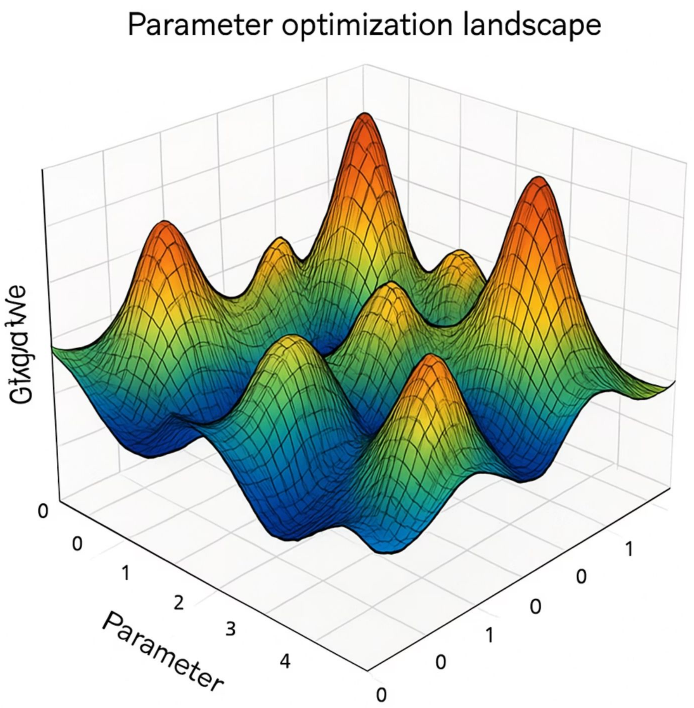
En Scikit-learn, existen modelos que integran la regularización de forma nativa, como Lasso, Ridge o ElasticNet. Estos modelos permiten ajustar un hiperparámetro clave: **alpha**, que controla la intensidad de la penalización. Cuanto mayor sea el valor de alpha, más se reducirá la magnitud de los coeficientes, lo que puede ayudar a evitar el sobreajuste pero también puede inducir subajuste si se exagera.

```
from sklearn.linear_model import Lassofrom sklearn.datasets import make_regressionfrom sklearn.model_selection import train_test_splitfrom sklearn.metrics import mean_squared_error# Generar datos sintéticosX, y = make_regression(n_samples=100, n_features=20, noise=0.5, random_state=42)X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)# Modelo con regularización L1 (Lasso)lasso = Lasso(alpha=0.1) # Ajustar alpha según necesidadlasso.fit(X_train, y_train)# Evaluar el modeloy_pred = lasso.predict(X_test)mse = mean_squared_error(y_test, y_pred)print(f"Error cuadrático medio: {mse:.4f}")# Ver cuántos coeficientes se redujeron a ceron_zeros = sum(lasso.coef_ == 0)print(f"Coeficientes eliminados: {n_zeros} de {len(lasso.coef_)}")
```



Impacto de la regularización

Este ejemplo ilustra cómo la regularización L1 puede ser implementada fácilmente para reducir la complejidad del modelo. Modificando el valor de alpha, puedes experimentar diferentes grados de penalización para observar su impacto en el desempeño. Esta práctica es fundamental en entornos donde se busca un equilibrio entre precisión y generalización.



Optimización de Parámetros

La optimización de parámetros es una etapa clave en el ciclo de vida de un modelo de aprendizaje automático, ya que permite mejorar su rendimiento al encontrar la mejor combinación de hiperparámetros. Estos no se aprenden directamente de los datos, por lo que deben ajustarse mediante experimentación y validación. Un mal ajuste puede conducir a modelos que generalizan mal, ya sea por subajuste o sobreajuste.

alkemy

Estrategias de optimización de parámetros

Búsqueda en grilla (Grid Search)

Una de las estrategias más utilizadas para optimizar hiperparámetros es la **búsqueda en grilla (Grid Search)**. Este método prueba todas las combinaciones posibles de un conjunto definido de valores. Aunque puede ser costoso computacionalmente, garantiza una evaluación exhaustiva y es especialmente útil cuando se tiene un espacio de búsqueda acotado y bien definido.

Búsqueda aleatoria (Random Search)

Otra alternativa es la **búsqueda aleatoria (Random Search)**, que selecciona combinaciones aleatorias de hiperparámetros dentro de los rangos establecidos. Este enfoque es más eficiente en problemas con muchos hiperparámetros o espacios grandes de búsqueda, y suele encontrar buenas soluciones con menos evaluaciones que Grid Search.

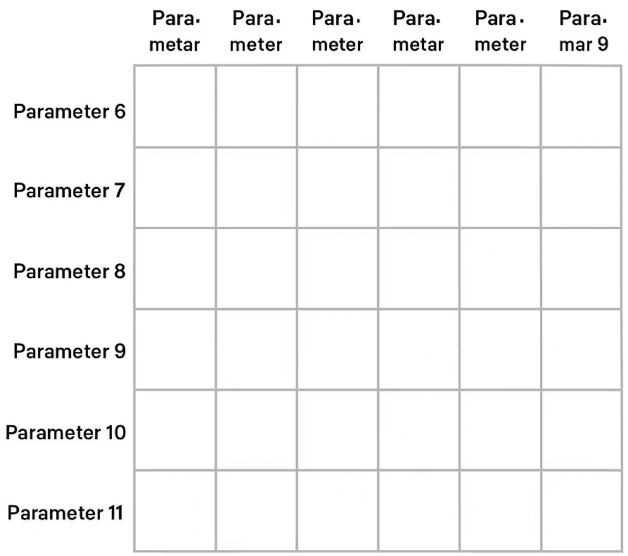
Optimización bayesiana

Para modelos complejos, se pueden emplear métodos más avanzados como la **optimización bayesiana**, que modela el rendimiento del modelo como una función probabilística y escoge nuevas combinaciones de hiperparámetros donde es más probable encontrar mejoras. Sin embargo, estos métodos requieren bibliotecas adicionales como Optuna o BayesianOptimization.

Ejemplo con GridSearchCV y Random Forest

```
from sklearn.ensemble import RandomForestClassifierfrom sklearn.model_selection import GridSearchCVfrom sklearn.datasets import make_classificationfrom sklearn.model_selection import train_test_split# Generar datos sintéticosX, y = make_classification(n_samples=1000, n_features=10, random_state=42)X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)# Definir modelo y grilla de hiperparámetrosrf = RandomForestClassifier(random_state=42)param_grid = {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20, 30], 'min_samples_split': [2, 5, 10], 'min_samples_leaf': [1, 2, 4]}# Búsqueda en grilla con validación cruzadagrid_search = GridSearchCV(rf, param_grid, cv=5, scoring='accuracy')grid_search.fit(X_train, y_train)# Mejores hiperparámetros y rendimientoprint(f"Mejores parámetros: {grid_search.best_params_}")print(f"Mejor puntuación: {grid_search.best_score_:.4f}")
```

alkemy



Grilla de Parámetros

La **grilla de parámetros** es una estructura que define el conjunto de valores que serán evaluados durante el proceso de optimización de hiperparámetros. Esta estrategia es implementada frecuentemente en la técnica de Grid Search, en la cual se combinan todos los valores posibles especificados en la grilla y se evalúa el desempeño del modelo con cada una de esas combinaciones.

alkemy

Características de la grilla de parámetros

Exploración sistemática

El objetivo de una grilla de parámetros es permitir una exploración sistemática del espacio de búsqueda. Por ejemplo, si definimos tres posibles valores para el número de árboles (n_estimators) y tres para la profundidad máxima (max_depth), Grid Search probará las nueve combinaciones posibles. Aunque este método garantiza una evaluación completa, también puede ser costoso computacionalmente si se amplía el número de valores o hiperparámetros.

Reproducibilidad

Una ventaja importante de la grilla de parámetros es su reproducibilidad. Al definir explícitamente cada combinación, los experimentos pueden ser replicados fácilmente, lo que resulta fundamental en contextos donde la trazabilidad y la validación de modelos son prioritarias. También permite registrar el rendimiento asociado a cada configuración, lo que facilita el análisis comparativo.

Desventajas

Sin embargo, su principal desventaja es la **ineficiencia computacional** cuando el número de combinaciones crece exponencialmente. Por esta razón, se recomienda utilizar grillas de tamaño manejable o, en su defecto, aplicar alternativas como búsqueda aleatoria (RandomizedSearchCV) o técnicas bayesianas más eficientes.

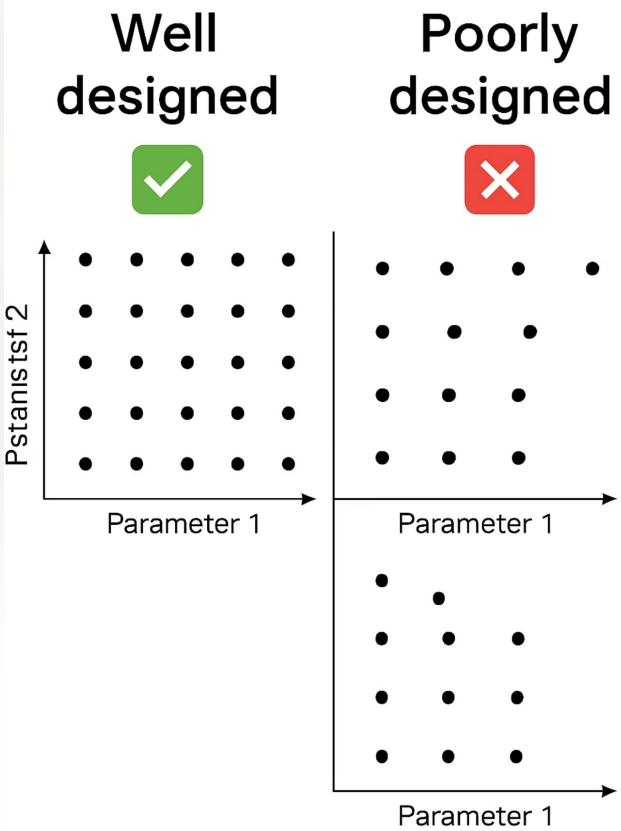
Ejemplo de grilla de parámetros con SVM

```
from sklearn.svm import SVCfrom sklearn.model_selection import GridSearchCVfrom sklearn.datasets import load_breast_cancerfrom sklearn.model_selection import train_test_splitfrom
sklearn.preprocessing import StandardScaler# Cargar datosX, y = load_breast_cancer(return_X_y=True)X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)# Escalar datos
(importante para SVM)scaler = StandardScaler()X_train_scaled = scaler.fit_transform(X_train)X_test_scaled = scaler.transform(X_test)# Definir modelo y grilla de hiperparámetrossvm =
SVC(random_state=42)param_grid = { 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.1, 0.01], 'kernel': ['rbf', 'linear', 'poly']}# Búsqueda en grilla con validación cruzadagrid_search =
GridSearchCV(svm, param_grid, cv=5, scoring='accuracy')grid_search.fit(X_train_scaled, y_train)# Mejores hiperparámetros y rendimientoprint(f"Mejores parámetros:
{grid_search.best_params_}")print(f"Mejor puntuación: {grid_search.best_score_:.4f}")
```

alkemy

Beneficios de una grilla bien diseñada

Este ejemplo muestra cómo una grilla bien diseñada permite probar diversas configuraciones del modelo y encontrar la que ofrece mejor rendimiento. La clave está en definir una grilla equilibrada que contemple suficiente variabilidad sin sacrificar eficiencia computacional. Esto es especialmente útil cuando se integran los modelos en sistemas de producción donde se exige robustez y trazabilidad.



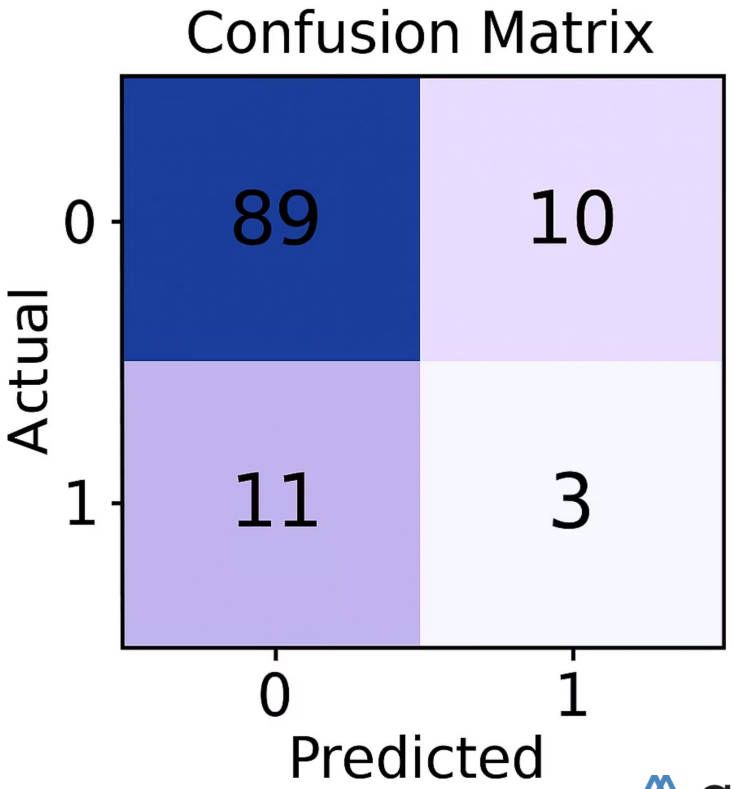
alkemy

Balanceo de Datos

En el aprendizaje automático supervisado, el **balanceo de datos** es esencial cuando se trabaja con conjuntos de datos desbalanceados, es decir, cuando las clases no están representadas de forma equitativa. Esto es común en casos de detección de fraudes, diagnóstico médico o análisis de abandono de clientes, donde una clase puede estar presente en el 90% de los casos y otra solo en el 10%. Los modelos tienden a favorecer la clase mayoritaria, lo que afecta negativamente la calidad de las predicciones.

Problemas del desbalanceo de clases

Uno de los efectos más problemáticos del desbalanceo es que un modelo puede mostrar una alta precisión global, pero en realidad no estar capturando correctamente la clase minoritaria. Por ejemplo, si solo el 5% de los datos corresponde a fraudes, un modelo que prediga "no fraude" en todos los casos tendrá 95% de precisión, pero no servirá para identificar fraudes reales. Por eso, se deben aplicar estrategias específicas para balancear el conjunto de entrenamiento.



alkemy

Enfoques para el balanceo de datos

Submuestreo

Reduce el número de muestras de la clase mayoritaria para equilibrar con la clase minoritaria.

Sobremuestreo

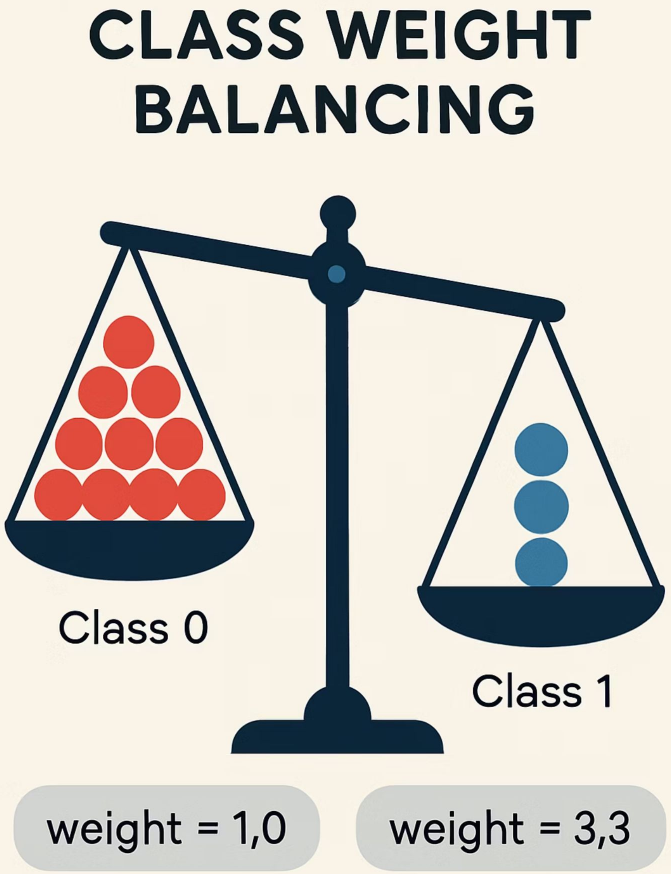
Aumenta las muestras de la clase minoritaria (replicando datos o generando datos sintéticos con técnicas como SMOTE).

Ponderación de clases

Ajusta la función de pérdida para que penalice más los errores cometidos sobre la clase minoritaria.

Implementación del balanceo en Scikit-learn

Scikit-learn facilita la implementación de estas estrategias. Algunos modelos como LogisticRegression, RandomForestClassifier o SVC permiten utilizar el parámetro class_weight='balanced' para asignar pesos automáticos inversamente proporcionales a la frecuencia de las clases. Esta es una forma simple y efectiva de mejorar el rendimiento del modelo sin alterar el conjunto de datos.



Ejemplo práctico con regresión logística

```
from sklearn.linear_model import LogisticRegressionfrom sklearn.datasets import make_classificationfrom
sklearn.model_selection import train_test_splitfrom sklearn.metrics import classification_report# Generar datos desbalanceados
(10% clase positiva, 90% clase negativa)X, y = make_classification(n_samples=1000, n_features=20, weights=[0.9, 0.1],
random_state=42)X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)# Modelo sin
balanceomodel_unbalanced = LogisticRegression(random_state=42)model_unbalanced.fit(X_train, y_train)y_pred_unbalanced =
model_unbalanced.predict(X_test)# Modelo con balanceo de clasesmodel_balanced = LogisticRegression(class_weight='balanced',
random_state=42)model_balanced.fit(X_train, y_train)y_pred_balanced = model_balanced.predict(X_test)# Comparar
resultadosprint("Modelo sin balanceo:")print(classification_report(y_test, y_pred_unbalanced))print("\nModelo con
balanceo:")print(classification_report(y_test, y_pred_balanced))
```

Live Coding

¿En qué consistirá la Demo?

Vamos a optimizar un modelo de clasificación paso a paso, incorporando selección de variables, ajuste de hiperparámetros y balanceo de clases.

1. *Cargar un dataset (churn.csv) con la columna objetivo: "Se va / No se va".*
2. *Realizar un análisis exploratorio breve para identificar variables relevantes.*
3. *Dividir los datos en entrenamiento y prueba.*
4. *Entrenar un modelo base con DecisionTreeClassifier sin optimización.*
5. *Aplicar SelectKBest para reducir dimensionalidad.*
6. *Usar GridSearchCV para ajustar hiperparámetros (max_depth, min_samples_split).*
7. *Aplicar class_weight='balanced' y comparar rendimiento.*
8. *Evaluar el mejor modelo con matriz de confusión y classification_report.*
9. *Visualizar los resultados de las diferentes combinaciones de parámetros.*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Optimizando un modelo real

Optimizando un modelo real

Contexto: 🙌

Trabajás en una empresa de telecomunicaciones que quiere predecir si un cliente abandonará el servicio (churn). El conjunto de datos contiene variables de consumo, interacción con el servicio y demografía.

Consigna: ✍️

1. Desarrollar un pipeline de clasificación que aplique las siguientes optimizaciones sobre un modelo de árbol:
2. Seleccionar variables relevantes (manual o automática).
3. Aplicación de GridSearchCV para buscar hiperparámetros óptimos (max_depth, min_samples_split, criterion).
4. Regularización con DecisionTreeClassifier y control de sobreajuste.
5. Aplicación de class_weight='balanced' para tratar desbalanceo.
6. Comparación de desempeño con y sin optimización.

Paso a paso: ⚙️

1. Cargar el dataset churn_telco.csv.
2. Preprocesar variables categóricas (encoding) y escalar numéricas.
3. Aplicar selección de variables (SelectKBest, VarianceThreshold o manual).
4. Entrenar un modelo base sin optimización.
5. Aplicar Grid Search con validación cruzada.
6. Evaluar desempeño y explicar por qué el modelo mejoró (o no).
7. Graficar resultados de cada configuración de hiperparámetros.

Tiempo 🕐**:** *45 Minutos*

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Aplicamos técnicas avanzadas de Feature Engineering.***
- ✓ ***Utilizamos regularización para evitar el sobreajuste.***
- ✓ ***Optimizamos hiperparámetros mediante Grid Search.***
- ✓ ***Balanceamos conjuntos desbalanceados.***
- ✓ ***Integramos estas técnicas en pipelines reproducibles.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy